

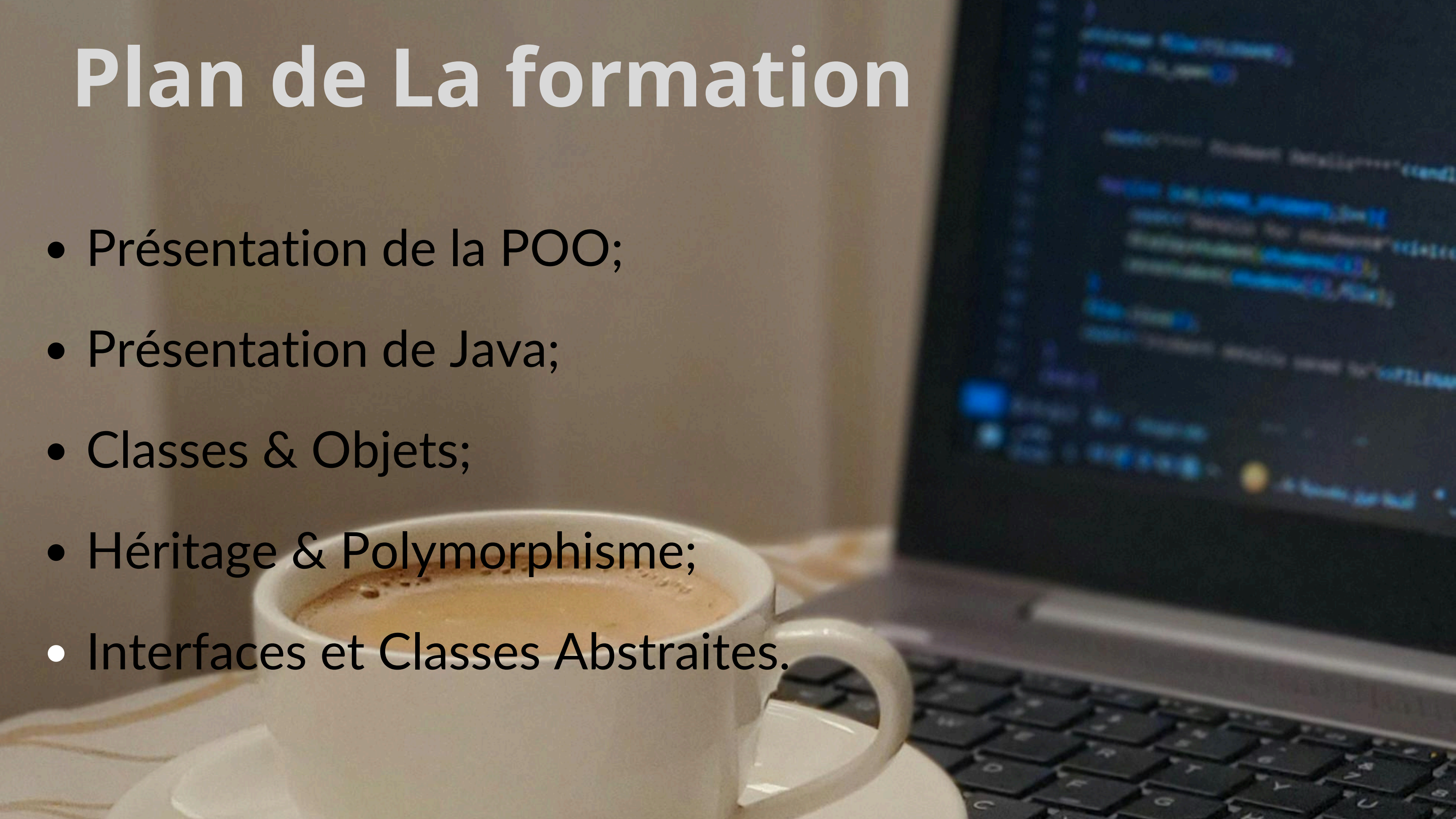
# Java Training





# Plan de La formation

- Présentation de la POO;
- Présentation de Java;
- Classes & Objets;
- Héritage & Polymorphisme;
- Interfaces et Classes Abstraites.





# C'est quoi la POO?

La POO est une manière de concevoir et d'organiser un programme en le basant sur des objets.

## OBJECT ORIENTED PROGRAMMING



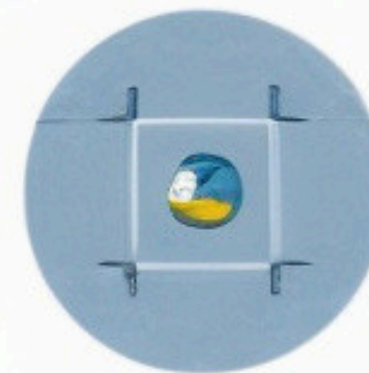
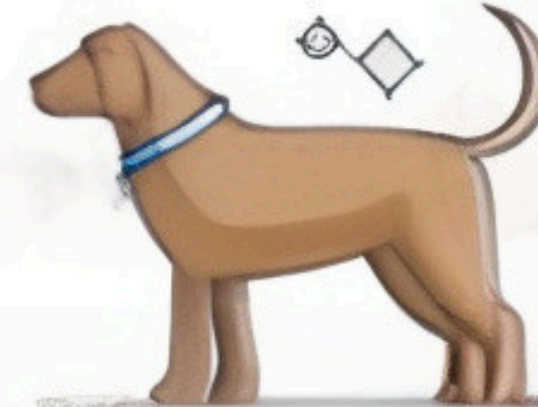
**CLASS**

○○○○○



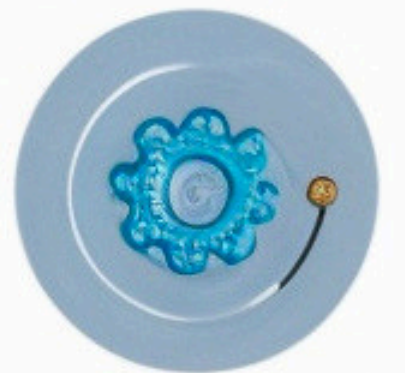
**OBJECTS**

○○○○○○○○



**ATTRIBUTES**

○ WEIGHT  
○ AGE  
○ POSITION



**METHODS**

○ BARK  
○ WAG  
○ RUN



# Pourquoi la POO?

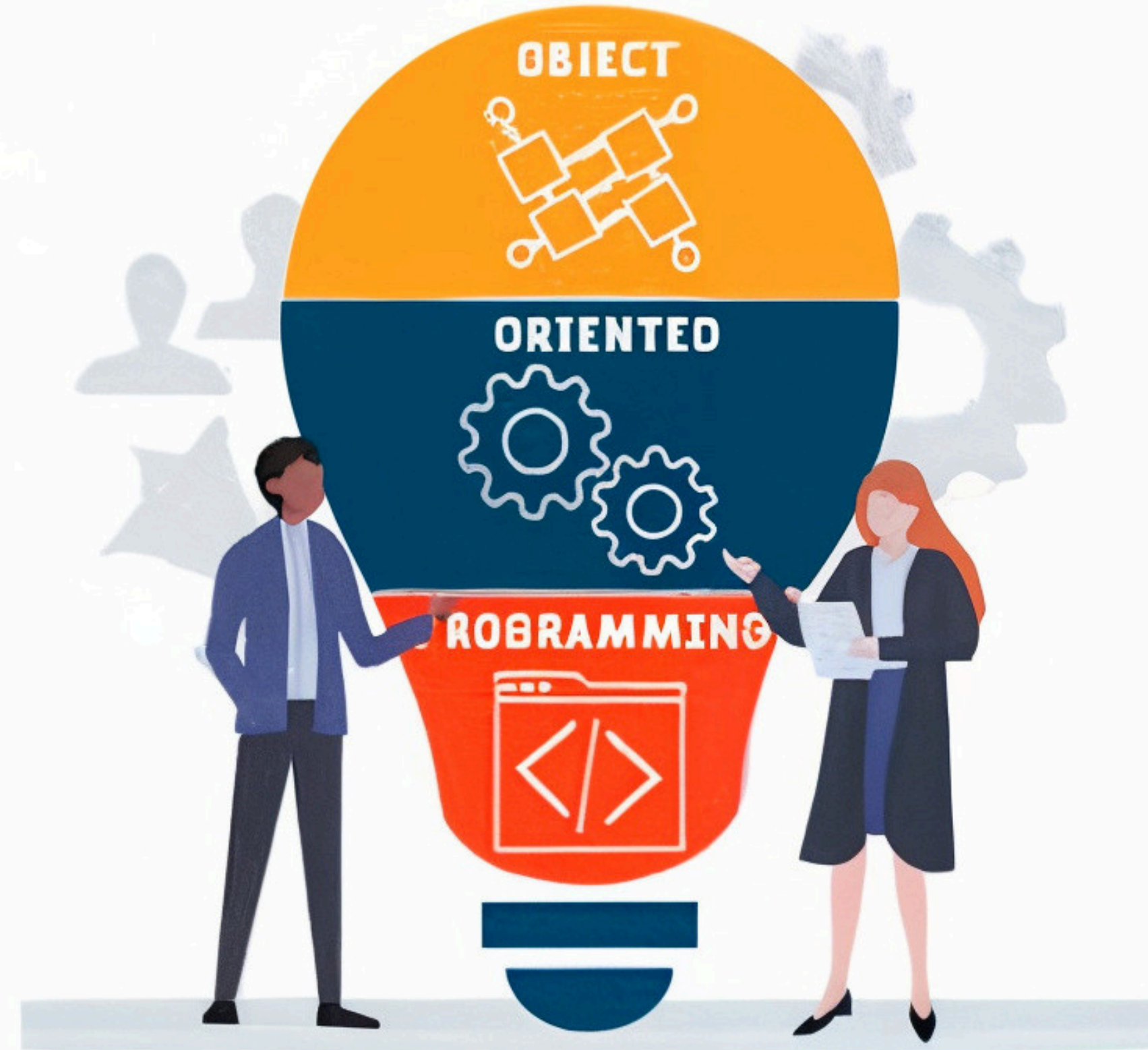
Meilleure  
organisation du  
code

Réutilisation  
du code

Maintenance  
facilitée

correspondance  
avec la réalité

Meilleure  
collaboration en  
équipe



# Pourquoi Java?

Write Once Run  
Anywhere

Sécurité &  
Simplicité

Réseaux et  
distributed  
computing(RMI )





# Interprété et Compilé

## Source Code

```
public class  
HelloWorld {  
    // ...  
}
```

Source Code



Compiler



.CLASS

Class File



JVM



**Versions LTS servant de points d'ancrage stables pour l'écosystème d'entreprise.**

**Versions non-LTS servant de transition rapide pour l'innovation**

| Version JDK |    | Publication | Statut de Support |  |
|-------------|----|-------------|-------------------|--|
|             | 11 | sept. 2018  | LTS               |  |
|             | 12 | mars 2019   | Standard (STS)    |  |
|             | 13 | sept. 2019  | Standard (STS)    |  |
|             | 14 | mars 2020   | Standard (STS)    |  |
|             | 15 | sept. 2020  | Standard (STS)    |  |
|             | 16 | mars 2021   | Standard (STS)    |  |
|             | 17 | sept. 2021  | LTS               |  |
|             | 18 | mars 2022   | Standard (STS)    |  |
|             | 19 | sept. 2022  | Standard (STS)    |  |
|             | 20 | mars 2023   | Standard (STS)    |  |
|             | 21 | sept. 2023  | LTS               |  |
|             | 22 | mars 2024   | Standard (STS)    |  |
|             | 23 | sept. 2024  | Standard (STS)    |  |
|             | 24 | mars 2025   | Standard (STS)    |  |
|             | 25 | sept. 2025  | LTS (Prévue)      |  |

# Hello World

|   |   |                                                          |
|---|---|----------------------------------------------------------|
| 1 | ▶ | <code>public class HelloWorld {</code>                   |
| 2 | ▶ | <code>    public static void main(String[] args){</code> |
| 3 | 💡 | <code>        System.out.println("Hello World");</code>  |
| 4 |   | <code>    }</code>                                       |
| 5 |   | <code>}</code>                                           |
| 6 |   |                                                          |

**javac HelloWorld.java**  
**java HelloWorld.java**



# Les types de variables

## Les types Primitifs

- short (2 octets)
- int (4 octets)
- long (8 octets)
- float (4 octets)
- double (8 octets)
- char
- boolean

## Les Objets

- String

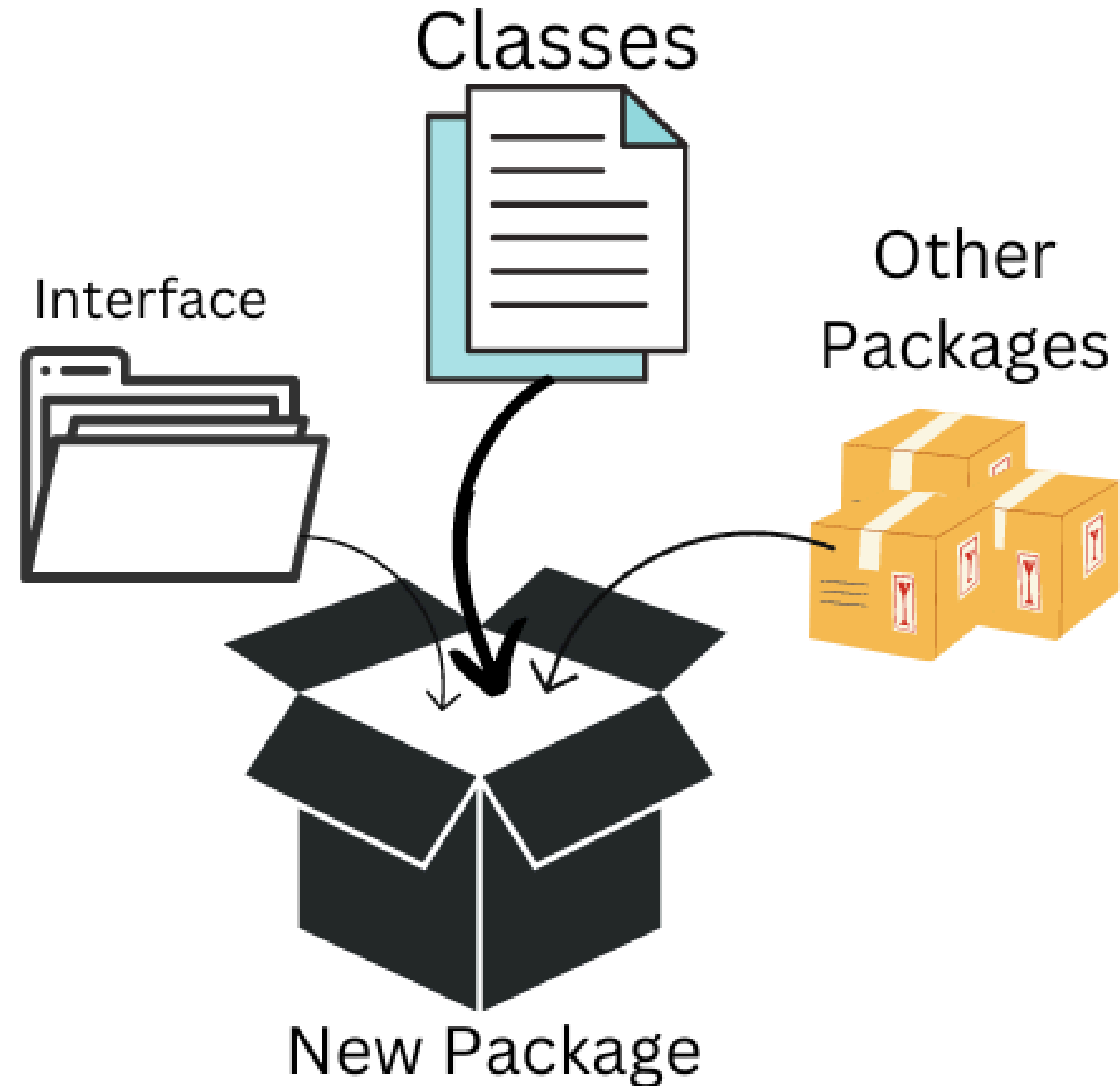
```
String string = new String()
```

```
String string = "chaine"
```



**une classe est un  
modèle représentatif  
des propriétés et  
comportements d'un  
objet**

**Un package est un  
ensemble regroupé de  
classe de  
fonctionnalités  
communes**



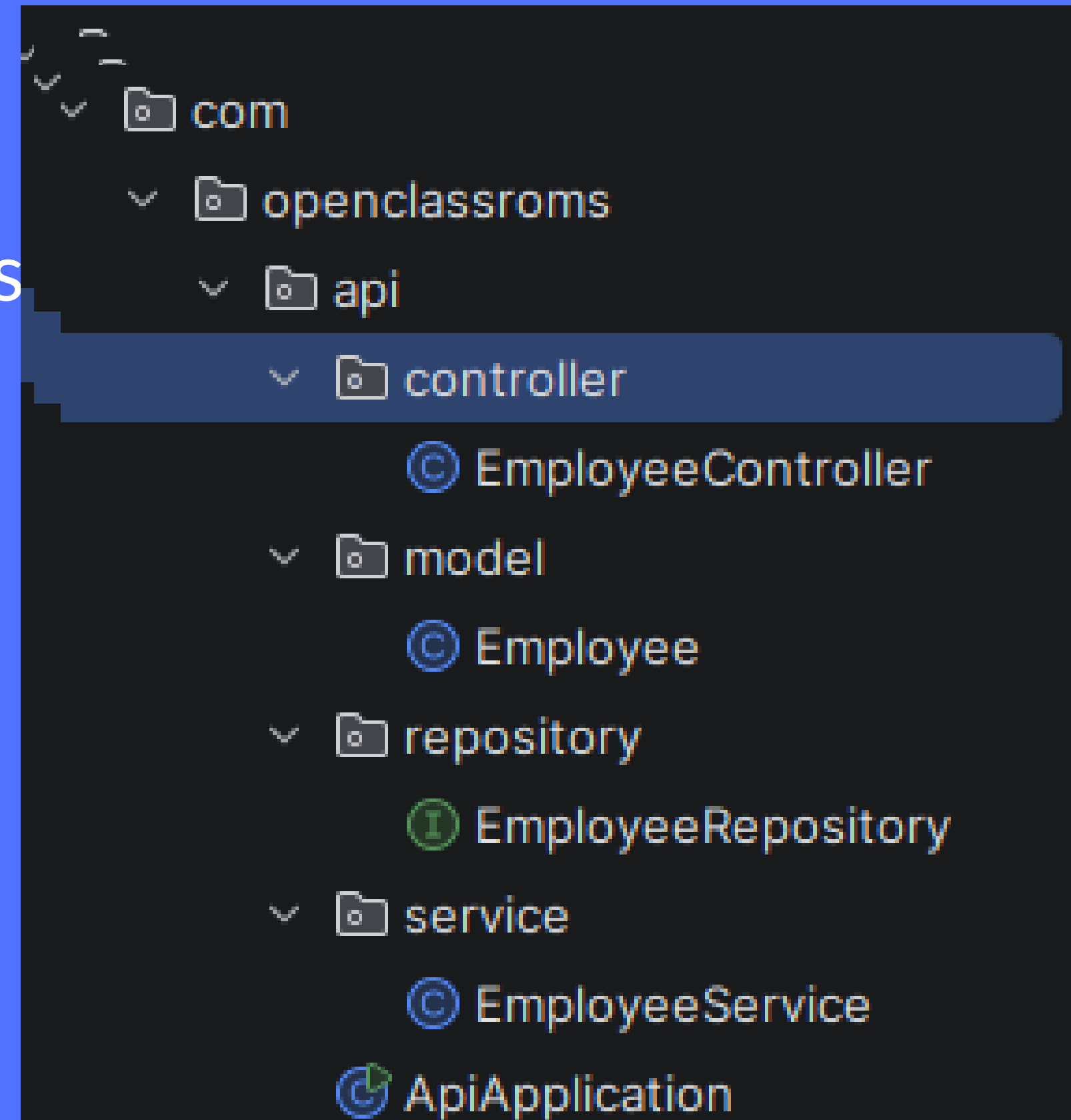
# Prise en main de l'Éditeur





# Conventions de nommage

- Le nom des packages commence par une lettre minuscule et celui des classes par une lettre majuscule
- Le nom du package est précédé d'une séquence comprenant le nom de domaine inversé de l'entreprise suivi du nom du projet et du package parent



# Visibilité des classes

- **public:** La classe est accessible depuis n'importe quelle autre classe;
- **protected:** La classe n'est accessible que depuis les sous-classes et les classes du même package;
- **final:** La classe ne peut-être étendue par une sous-classe;



# Visibilité des attributs

- public: L'attribut est accessible depuis n'importe quelle autre classe;
- private: L'accès à l'attribut est restreint à sa propre classe;
- protected: L'attribut n'est accessible que depuis la classe d'origine, les classes du même package et également les sous-classes.
- final: Attribut constant dont la valeur ne peut être modifié après initialisation;

# Visibilité des méthodes

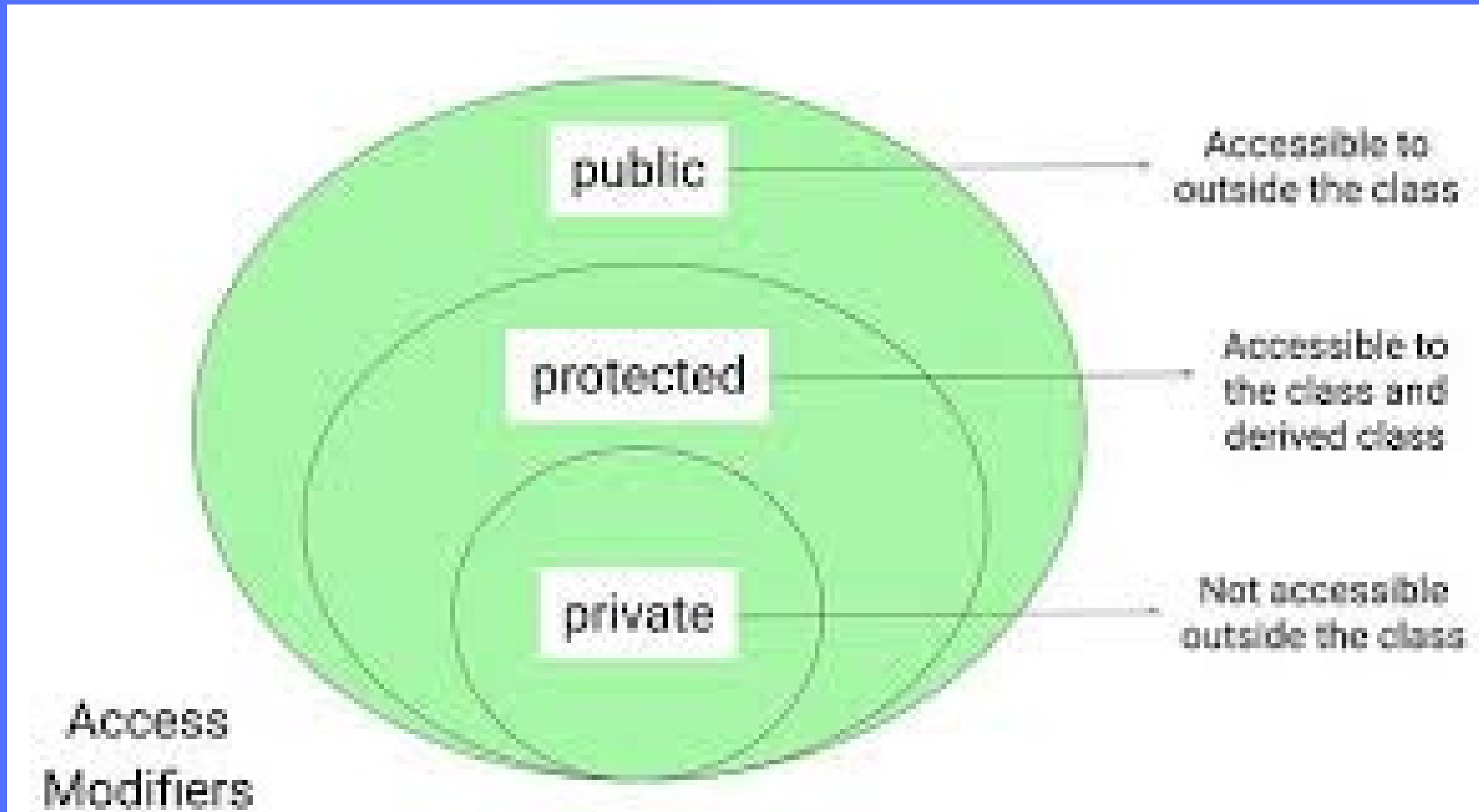
- **public:** La méthode est accessible depuis n'importe quelle autre classe peut importe le package;
- **private:** La méthode n'est accessible que depuis sa propre classe;
- **protected:** La méthode n'est accessible que depuis la classe d'origine, les classes du même package et également les sous-classes
- **final:** La méthode ne peut-être redéfinie par une sous-classe;
- **static:** La méthode appartient à la classe et non à une instance spécifique de celle-ci.



# Quelques méthodes spécifiques

- Constructeur: Méthode spéciale qui est appelée automatiquement lors de la création d'un nouvel objet d'une classe;
- Assesseur (getter): Méthode publique qui permet d'obtenir la valeur d'un champ privé d'une classe;
- Mutateur (setter): Méthode publique qui permet de modifier la valeur d'un attribut privé d'une classe.

- Les getters & setters permettent l'encapsulation des données
- L'encapsulation est le regroupement des données suivant un ensemble de routines permettant d'en contrôler l'accès et d'en protéger l'intégrité.





# Types de Variables dans une classe

- Les variables d'instance : ce sont elles qui définissent les caractéristiques des objets;
- Les variables de classe : celles-ci sont communes à toutes les instances d'une classe;
- Les variables locales : ce sont des variables utilisées pour travailler au sein des objets.



# Héritage & Polymorphisme



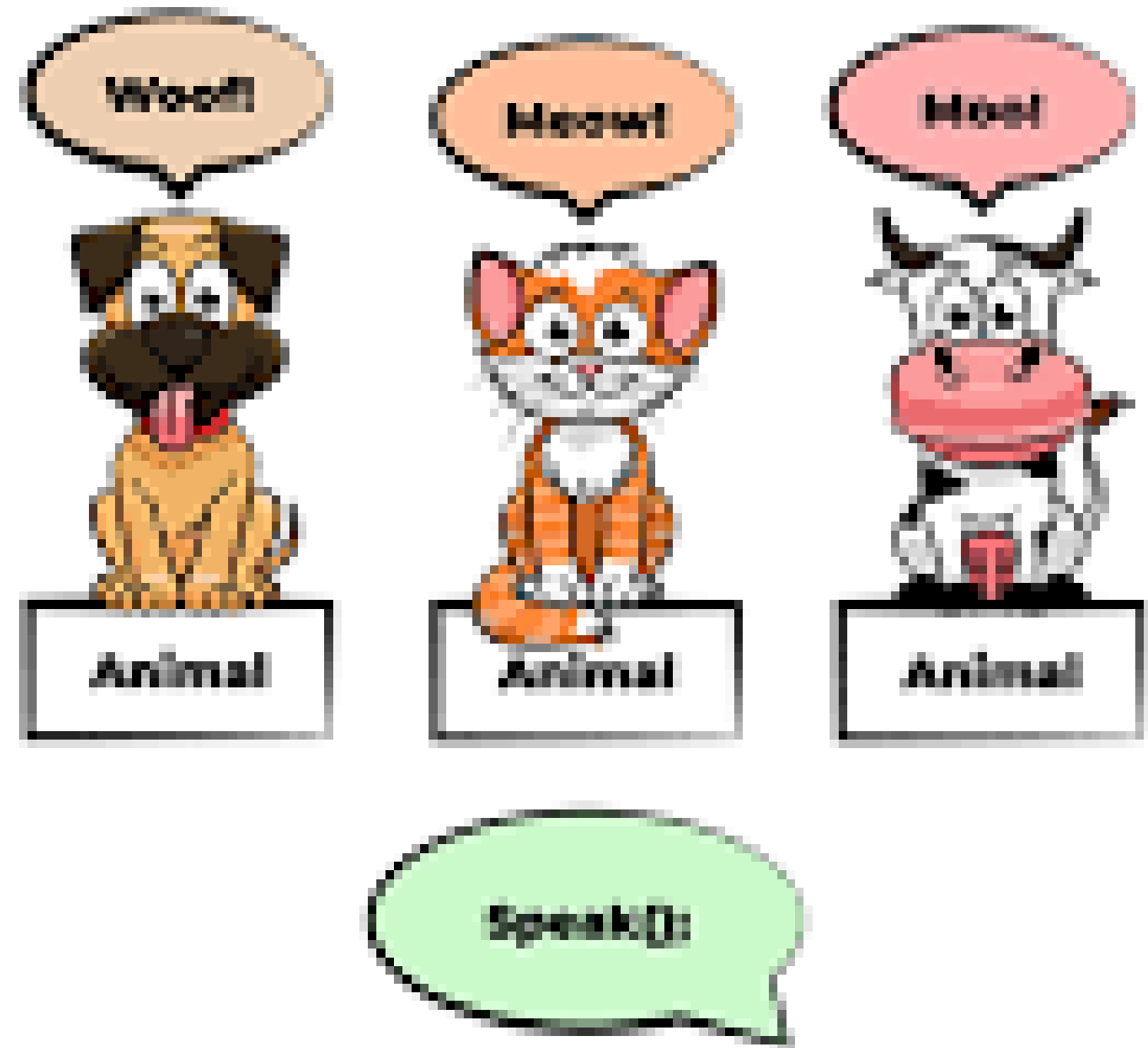
# Héritage

- C'est un concept de la POO où une nouvelle classe (classe fille) hérite des propriétés d'une classe existante (classe mère) à l'aide du mot clé **extends**;
- Le mot clé **super** permet d'appeler une méthode de la classe mère au sein de la classe fille.



# Polymorphisme

Le polymorphisme est la  
capacité donnée à une même  
opération de s'exécuter  
différemment  
suivant le contexte de la  
classe où elle se trouve



# Interfaces & Classes abstraites



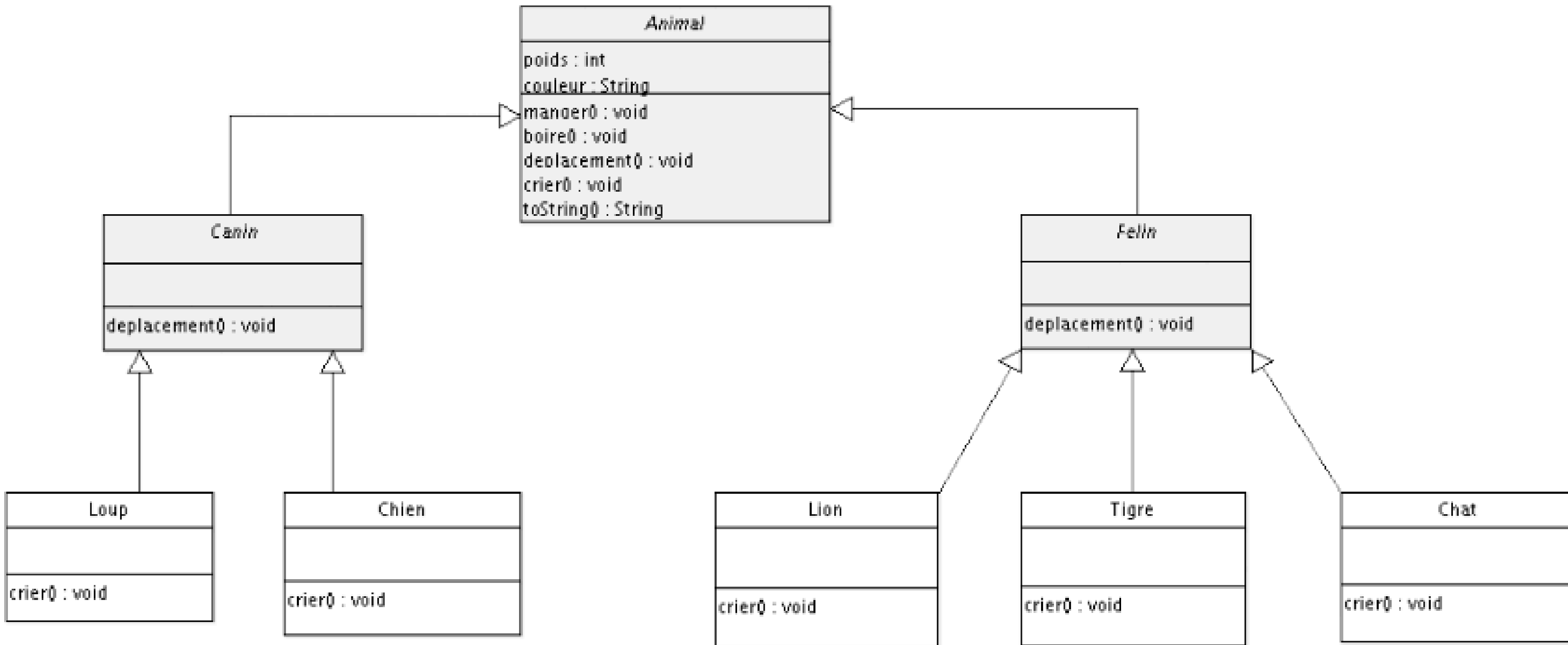


# Les Classes Abstraites

- Une classe abstraite est une classe non instanciable destinée à l'extension et pouvant contenir des méthodes abstraites (sans corps).
- Les Classes et méthodes abstraites sont déclarée avec le mot clé **abstract**.



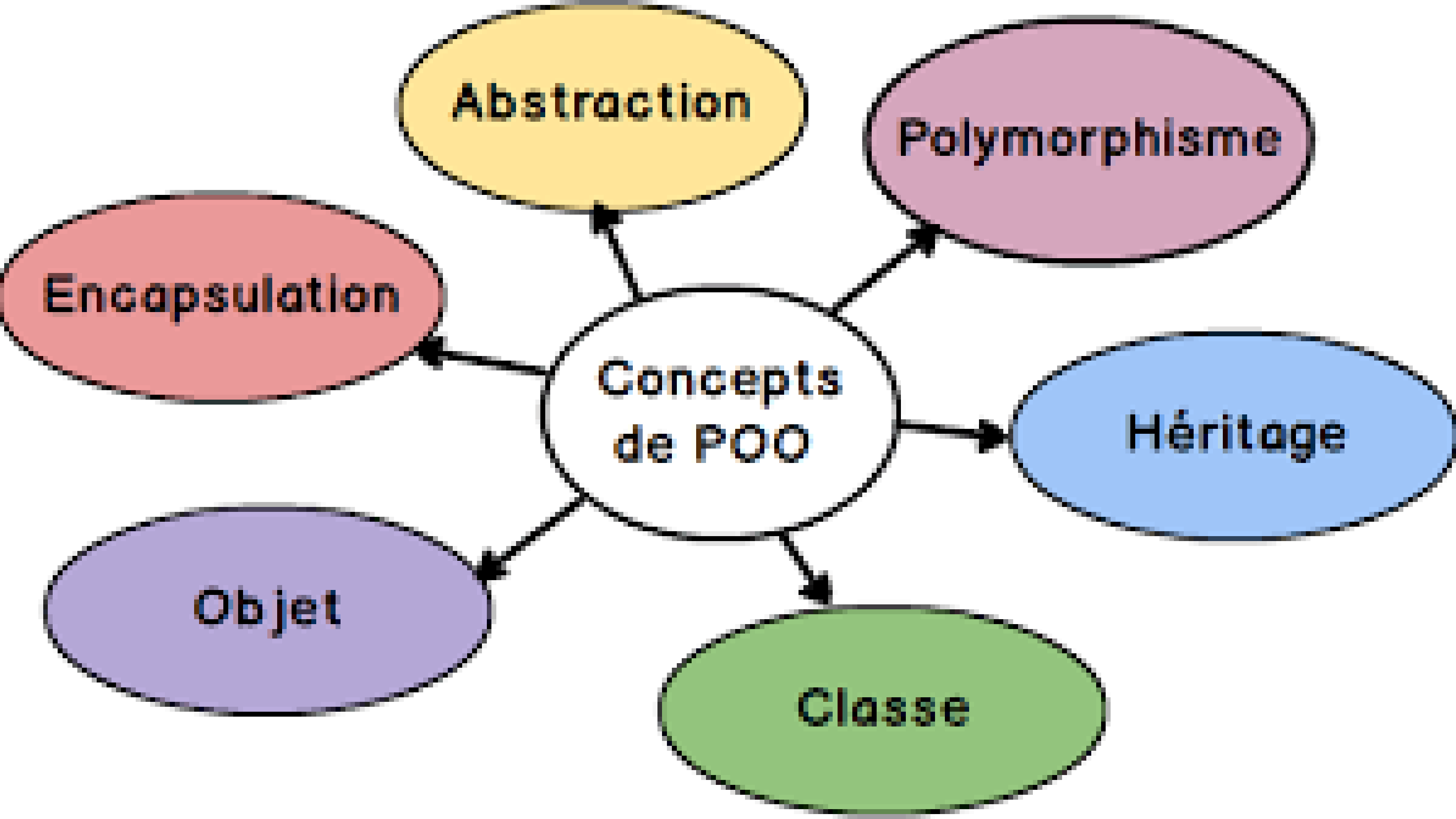
# Identifier la classe abstraite et pratiquer



# Les Interfaces

- Une interface définit un ensemble de méthodes abstraites(sans corps) et de constantes qu'une classe peut implémenter et fournir obligatoirement une implémentation pour ces méthodes;
- Les interfaces sont créées avec le mot clé **interface** et implémentées en utilisant le mot clé **implements**.





**Merci pour votre  
aimable attention**

